

GatKeeper

AI-Native Security Intelligence Platform

Project Analysis & Architecture Document

Version 1.0 | June 2026

Author: Karla Albert <karlalbert152@gmail.com>

Status: Phase 1 — MVP Architecture

CONFIDENTIAL

Table of Contents

1. Executive Summary	3
2. Problem Statement & Market Analysis	4
3. Solution & Strategic Positioning	5
4. Architecture — 4-Layer Design	6
5. Tech Stack & Design Justifications	7
6. Phase 1 MVP — Directory Structure	8
7. Multi-Agent Analysis System	9
8. Subconscious Engine Deep Dive	10
9. Competitive Analysis	11
10. .gat Report Format Specification	12
11. Phase 1 Roadmap (3 Months)	13
12. Monetization & Go-to-Market	14

1. Executive Summary

GatKeeper is a next-generation code intelligence platform designed to address one of the most critical challenges of our digital era: software security at scale. Unlike existing tools that only detect problems already present in the code, GatKeeper adopts a radically different approach: anticipate, simulate, and predict vulnerabilities before they exist.

The platform combines a multi-agent architecture based on specialized AI models, an autonomous simulation engine (the Subconscious Engine), and an evolving codebase memory to deliver unprecedented security coverage. GatKeeper targets both individual developers and large organizations (Fortune 500, financial institutions, governments) with strict compliance requirements (GDPR, SOC2, ISO27001, HIPAA).

Key Metrics

Metric	Value	Context
Scenarios simulated / night	48,000+	By Subconscious Engine
Specialized AI agents	6 agents	Each fine-tuned on its domain
Temporal projection	30 days	Prospective code simulation
Supported languages	15+	C, C++, Rust, Python, JS, Go...
Addressable market	\$25B	Global software security 2025
Estimated client savings	80%	vs. traditional external audit

2. Problem Statement & Market Analysis

2.1 The Silent Crisis of Software Security

We live in a paradoxical era: never have development tools been so powerful, and yet software security vulnerabilities are reaching record levels. In 2024, the average cost of a data breach exceeded \$4.5 million per incident. Large organizations face multiple structural problems simultaneously:

- Massive codebases (millions of lines) impossible to manually audit with rigor
- Dozens of teams developing in parallel creating unforeseen interactions between modules
- External dependencies (npm, crates.io, PyPI) introducing third-party vulnerabilities at each update
- Expensive external security audits (\$100K-\$500K per audit) that are infrequent
- Fragmented tools that don't communicate: SAST, DAST, SCA, secret scanners — each in its silo
- Increasingly stringent regulatory compliance (GDPR, SOC2, HIPAA) with sanctions

2.2 Limitations of Current Tools

Existing solutions suffer from a fundamental flaw: they are reactive. They find what is already broken, never what will be. They analyze syntax and some known patterns, but do not understand the code as a whole:

- SonarQube and SAST tools: static analysis without semantic understanding, high false positive rate
- Snyk and SCA tools: focused on dependencies, ignoring internal logic
- Manual pentest tools: expensive, episodic, not integrated into the development cycle
- GitHub Copilot and AI assistants: suggest code but do not audit in depth
- No tool combines evolutionary memory, prospective analysis, and multi-layer analysis

3. Solution & Strategic Positioning

3.1 Founding Vision

GatKeeper is the first living code intelligence platform — a system that learns the DNA of your codebase, simulates thousands of possible futures in the background, and proactively alerts before problems arrive. Like a doctor who predicts disease before symptoms rather than treating it once declared.

GatKeeper unifies in a single platform what today requires 5 to 8 distinct tools: static analysis, dependency analysis, secret scanning, threat modeling, automatic compliance, load simulation, and attack simulation.

3.2 Strategic Positioning

Dimension	Classic SAST	AI Assistants	GatKeeper
Approach	Reactive	Assistive	Predictive & proactive
Understanding	Syntactic	Partial contextual	Semantic + intentional
Project memory	'	'	'
Future simulation	'	'	'
100% on-premise	Partial	'	'
Multi-agents	'	'	'
Auto compliance	Partial	'	'

4. Architecture — 4-Layer Design

The GatKeeper platform is structured around four main layers, each with clearly defined responsibilities and integrated components from the technical stack.

[illegible]

Component Responsibilities

- **INTERFACE:** Entry point for developers. CLI for terminal workflows, IDE plugins for real-time feedback, Dashboard for team visibility.
- **INTEGRATION:** Automates security checks in the development cycle. Every commit triggers analysis.
- **INTELLIGENCE:** Core brain. Agents analyze, Subconscious simulates, DNA remembers, Tree-sitter parses.
- **DATA:** Persistent storage. SQLite for local/portable, PostgreSQL for enterprise scale, CVE databases for threat intelligence.

5. Tech Stack & Design Justifications

Component	Technology	Version	Justification
CLI main	Rust	1.75+	Native performance, memory safety guarantees
Code parsing	Tree-sitter	0.22+	Multi-language (15+), robust, incremental, embeddable
Agent orchestration	Tokio (async Rust)	1.35+	Millions of concurrent tasks, zero overhead
Local LLMs	Ollama	Latest	Zero API cost for nightly subconscious, absolute privacy
Cloud LLMs	Anthropic Claude	claude-3	Superior analysis quality for paid tiers
Web dashboard	React + TypeScript	18+/5+	Rich real-time UI via WebSockets
API backend	Axum (Rust)	0.7+	Consistent with CLI, excellent HTTP performance
Local storage	SQLite	3.45+	Lightweight, portable, versionable, no server needed
Enterprise DB	PostgreSQL	16+	Scalability, transactions, full-text search
CVE analysis	OSV + NIST NVD API	Latest	Official databases, open source, real-time updates
Attack sandbox	gVisor + Docker	Latest	Kernel-level isolation for secure PoC execution
Real-time comms	WebSockets	Latest	Live dashboard, immediate push alerts
Report format	TOML	1.0	Human-readable, machine-parseable, Git-versionable

6. Phase 1 MVP — Directory Structure

The Phase 1 MVP directory structure follows a Cargo workspace pattern with clearly separated crates for each major component.

```
gatkeeper/
% % % Cargo.toml
% % % rust-toolchain.toml
% % % LICENSE
% % % README.md
% % % .gitignore
%
% % % crates/
% % % % % gatkeeper-cli/
% % % % % % % src/
% % % % % % % main.rs
% % % % % % % commands/
% % % % % % % output.rs
% % % % % gatkeeper-core/
% % % % % % % src/
% % % % % % % config.rs
% % % % % % % error.rs
% % % % % % % project.rs
% % % % % % % report/
% % % % % gatkeeper-parser/
% % % % % % % src/
% % % % % % % languages/
% % % % % % % ast.rs
% % % % % % % walker.rs
% % % % % gatkeeper-agents/
% % % % % % % src/
% % % % % % % orchestrator.rs
% % % % % % % traits.rs
% % % % % % % security/
% % % % % % % secrets/
% % % % % % % supply_chain/
% % % % % gatkeeper-subconscious/
% % % % % % % src/
% % % % % % % engine.rs
% % % % % % % scheduler.rs
% % % % % % % threat/
% % % % % % % chaos/
% % % % % % % evolution/
% % % % % gatkeeper-dna/
% % % % % % % src/
% % % % % % % fingerprint.rs
% % % % % % % patterns.rs
% % % % % % % invariants.rs
% % % % % % % baseline.rs
% % % % % % % store.rs
%
% % % gatkeeper-lenses/
% % % migrations/
% % % templates/
% % % tests/
% % % scripts/
% % % docs/
% % % .github/workflows/

# Workspace root
# Main CLI binary
# Entry (clap)
# scan, report, init, status, whatif
# Terminal formatting
# Shared business logic
# Project config
# Error types
# Project detection
# .gat, risk_score, findings
# Tree-sitter multi-language
# Rust, Python, JS, C, Go
# Unified AST wrapper
# AST walking patterns
# 6 specialized AI agents
# Multi-agent orchestration
# Agent trait definition
# SQL/XSS, crypto, OWASP
# Hardcoded secrets detector
# Dependency CVE checking
# Subconscious Engine
# Main loop (Tokio spawn)
# Cron-like scheduling
# Rouge: attacker simulation
# Jaune: failure injection
# Verte: code generation
# DNA Engine (long-term memory)
# Cryptographic fingerprint
# Architectural pattern detection
# Code invariants
# Git behavioral baseline
# SQLite persistence
# Intent comprehension
# SQLite schema
# .gat template, config example
# Integration tests + fixtures
# PDF generation, CI helpers
# Documentation + PDFs
# CI/CD pipeline
```


7. Multi-Agent Analysis System

Six independent AI agents, each fine-tuned on its specific domain, work in parallel on each analysis. This multi-agent architecture enables depth impossible with a single generalist model.

Agent	Domain	Example Detections
SecurityAgent	Vulnerabilities & attacks	SQL/XSS/LDAP injection, timing attacks, weak crypto, OWASP T
LogicAgent	Logic bugs & coherence	Race conditions, deadlocks, edge cases, inter-module inconsi
PerformanceAgent	Efficiency & resources	Memory leaks, N+1 queries, unnecessary allocations, hot path
ComplianceAgent	Regulatory compliance	GDPR Art.32, SOC2, ISO27001 A.12, HIPAA HITECH
SecretAgent	Secrets & credentials	Hardcoded API keys, exposed JWT tokens, plaintext passwords
SupplyChainAgent	Supply chain security	Typosquatting, abandoned deps, third-party CVEs, licenses

The AgentOrchestrator manages parallel execution via Tokio async runtime, collecting findings from all agents and aggregating them into a unified report with calculated risk scores.

8. Subconscious Engine Deep Dive

8.1 Concept & Philosophy

The Subconscious Engine is the most innovative feature of GatKeeper and the one that radically distinguishes it from everything else on the market. Inspired by the human brain which continues to process information during sleep — consolidating learning, solving complex problems, establishing non-obvious connections — the Subconscious Engine operates in the background 24/7.

8.2 Three Layers of the Subconscious

- ROUGE (Red) — Threat Simulation: Simulates real attackers operating against your system. Models different adversary profiles and calculates most probable attack paths.
- JAUNE (Yellow) — Chaos Engineering: Simulates adverse conditions to identify hidden breaking points. Inspired by Netflix Chaos Engineering and Google SRE practices.
- VERTE (Green) — Code Evolution: Generates and evaluates improved alternatives to existing code. Acts as an AI pair-programmer working while the team sleeps.

8.3 Simulation Cycles

Phase	Frequency	Duration	Content
Light Scan	Every hour	~2 min	New commits, critical alerts, secret check
Nightly Simulation	Every night	~45 min	7-day projection, DNA update, KB correlation
Deep Simulation	Weekly	~6 hours	30-day projection, full architecture re-evaluation
Grand Audit	Quarterly	~24 hours	Exhaustive simulation, compliance report, strategic recs

9. Competitive Analysis

Feature	SonarQube	Snyk	Veracode	Semgrep	GatKeeper
Static analysis (SAST)	'	Partial	'	'	'
Dependency analysis (SCA)	Partial	'	'	Partial	'
Secret scanning	Partial	'	Partial	'	'
Multi-agent AI	'	'	'	'	'
Evolving memory (DNA)	'	'	'	'	'
Subconscious Engine 24/7	'	'	'	'	'
30-day simulation	'	'	'	'	'
Attack sandbox (PoC)	'	'	Partial	'	'
What-If mode	'	'	'	'	'
Auto compliance	Partial	Partial	'	'	'
Collective KB	'	'	'	'	'
Cross-repository	'	Partial	'	'	'
Post-quantum prep	'	'	'	'	'
100% on-premise	'	'	'	'	'
Versioned report	Partial	'	'	'	'

10. .gat Report Format Specification

The .gat file is far more than a simple output report. It is a structured, versionable, and machine-exploitable artifact that documents the complete health state of a project at a given moment. Designed to be attached to Git commits, integrated into CI/CD pipelines, and consumed by third-party tools via API.

```
[session]
organisation = "TechCorp SA"
project = "payment-service"
version = "2.4.1"
timestamp = "2026-06-03T08:00:00Z"
agents_utilises = ["SecurityAgent", "LogicAgent", "ComplianceAgent"]
lignes_analysees = 847_293
scenarios_sub = 48_293
duree_analyse = "6h32m (background)"

[risk_score]
global = 73/100
securite = 61/100
performance = 88/100
compliance = 79/100
tendance = "down (-7pts sur 14 jours)"

[[findings.critical]]
id = "GAT-CRIT-001"
agent = "SecurityAgent"
fichier = "src/auth/token.rs"
lignes = [234, 267]
type = "timing_attack"
cvss_score = 8.7
description = "Non-constant-time token comparison, exploitable via timing"
correction = "Use subtle::ConstantTimeEq"
effort_correction = "15 minutes"

[[subconscious.dreams]]
id = "DREAM-001"
couche = "threat"
decouverte_a = "03:47 AM"
horizon = "23 jours"
probabilite = 94
impact = "unauthorized admin access"
condition = "traffic > 10k req/s"

[subconscious.intuitions]
items = [
  "auth/ and payments/ diverging silently. Incompatible in 2 sprints.",
  "crypto-lib abandoned. CVE probability 60d: 71%."
]
```

11. Phase 1 Roadmap (3 Months)

Month 1 — Foundation

- Set up Cargo workspace with all crate structure
- Implement CLI with clap (init, scan, report, status commands)
- Integrate Tree-sitter parser for 5 languages (Rust, Python, JS, C, Go)
- Implement SecurityAgent basic patterns (injection, crypto, OWASP)
- Implement SecretAgent with hardcoded credential detection
- Generate .gat report in TOML format
- SQLite schema for DNA Engine storage

Month 2 — Intelligence

- Complete all 6 agents with pattern-based analysis
- Implement AgentOrchestrator with Tokio parallel execution
- Basic Subconscious Engine with hourly cycle and 24h projection
- DNA Engine v1: fingerprinting, baseline, pattern detection
- SupplyChainAgent with OSV/NVD API integration for CVE checking
- GitHub Actions CI/CD integration

Month 3 — Polish & Launch

- Plugin Neovim basic integration
- Comprehensive documentation and getting-started guide
- Integration test suite with vulnerable project fixtures
- Performance optimization and error handling hardening
- README, contributing guide, and community setup
- Public GitHub repository launch

12. Monetization & Go-to-Market

Pricing Plans

Plan	Target	Price	Key Features
Community	Individual devs, OSS	Free	1 project, CLI, basic SecurityAgent, .gat, 24h subconsc.
Pro	Freelancers, startups	€49/mo	Unlimited projects, 6 agents, dashboard, CI/CD, 7-day sub.
Business	Teams 5-50	€299/mo/team	Compliance, sandbox PoC, Horizon, KB, 30-day subconsc.
Enterprise	Large orgs 50+	\$50K-500K/yr	On-premise, fine-tuning, Telepathy, Quantum, SLA, support

Go-to-Market Strategy

- Phase 1: Community open source !' organic adoption by developers, GitHub contributions
- Phase 2: Product-led growth !' natural Pro upgrade via feature gates
- Phase 3: Sales-led growth !' commercial team targeting CISOs of large enterprises
- Strategic partners: integrators (Accenture, Deloitte), cloud providers (AWS, Azure, GCP)
- Conferences: DEF CON, Black Hat, RSA Conference, KubeCon for security credibility

GatKeeper

The first security tool that thinks.

Document generated by GatKeeper Project Analysis
Version 1.0 — June 2026

"While you slept, I found 3 things you need to know."